

Fundamentals of OOP for a single class

Is OOP an appropriate paradigm for solving complex problems?

SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ B3.1.1 Evaluate the fundamentals of OOP
- ▶ B3.1.2 Construct a design of classes, their methods and behaviour
- ▶ B3.1.4 Construct code to define classes and instantiate objects
- ▶ B3.1.5 Explain and apply the concepts of encapsulation and information hiding in OOP
- ▶ B3.1.3 Distinguish between static and non-static variables and methods

B3.1.1 Fundamentals of OOP

◆ **Object-oriented programming:** a form of programming that involves creating code for classes of objects, allowing many such objects to be created from a single code base, achieving a more modular and extensible software development process. It is like the idea of producing architectural blueprints, from which many similar houses can be constructed.

Object-oriented programming (OOP) is a paradigm in Computer Science that increases modularity by providing a new type of abstraction.

Just as you have experienced the benefits of modularity through writing functions that allow you to standardize behaviour for a given combination of input parameters, OOP lets you take this even further. It allows you not only to standardize behaviour, but also to standardize the structure of the data within your code.

OOP achieves this level of modularity by empowering you to define custom data types, known as “classes”. You are no longer constrained to basic data types like integers, floats and strings; you can now craft custom variables of types such as `Person`, `Book`, `BankAccount` or `ShoppingBasket`. Similar to creating various instances of integers or strings, you can instantiate numerous instances of your classes.

When you manage existing data types like strings and integers, they come with standardized methods for setting their values and for manipulation. For example, with strings, you don’t need to program custom functions to extract substrings, locate the first occurrence of a character or convert to uppercase or lowercase. These capabilities are inherent to the data type. With OOP, you tailor functionality specifically for your class, creating methods that align with the nature of the data it represents. Furthermore, similarly to using strings and integers without knowledge of their internal storage mechanisms, the internal data structure of your classes remains encapsulated, hidden behind the interface you provide as the creator of the class.

TOK

Are some types of knowledge less open to interpretation than others?

The structured nature of OOP, with its defined classes and behaviours, might suggest it is less open to interpretation than more flexible paradigms like procedural programming.

■ The advantages and disadvantages of using OOP in various programming scenarios

There are numerous advantages to using OOP in your programming, including:

- **Increased modularity:** Designing programming code around data and the functions that manipulate it can make it easier to manage large code bases. Objects can be created and modified independently of each other.
- **Code reusability:** A class, once written, can be imported into other projects and reused many times.
- **Encapsulation:** Hiding the internal mechanisms through which a class manages its internal data means that programmers only interact with the class through its well-defined interfaces, such as publicly exposed functions, and don't attempt to access the internal data directly. Programmers trust the class to know how to update its own internal data. This helps prevent unintended consequences resulting from managing data directly.
- **Scale:** By allowing increased modularity and reusability, OOP allows projects to scale in size yet remain maintainable.
- **Collaboration:** Increased modularity also increases the ease for delegating different parts of the project to different team members, allowing more people to work on the same project with ease.

It is worth acknowledging that OOP is not a perfect solution to all programming problems. There are some potential drawbacks to be aware of, including:

- **Learning curve:** OOP-related concepts, such as classes, objects and inheritance, can be difficult to understand for beginner programmers.
- **Increased complexity:** Small problems where a procedural approach would suffice can become unnecessarily complex to implement in a purely OOP approach. This complexity can also make projects more challenging to debug and maintain.
- **Overuse:** OOP is good at solving certain types of problems, but it is not a one-size-fits-all solution. Attempting to force everything to be viewed as an object of a class can result in deep inheritance hierarchies, leading to code that can be difficult to understand and maintain.
- **Overhead:** OOP will typically introduce additional overhead in time and space requirements compared to using paradigms with less abstraction.
- **Lack of optimization:** The focus of OOP is on providing abstractions to improve modularity. That comes at a cost in terms of organizing code into constructs that are more efficient for the CPU. In performance-critical applications, lower-level paradigms may be more suitable.
- **Object-centric design limitations:** OOP's focus on objects can sometimes lead to design limitations when dealing with certain problem domains. Some problems may be better modelled using alternative paradigms, such as functional programming or procedural programming.

So, while an important and valuable tool for any programmer, OOP is not a magic solution to all programming needs.

Common mistake

Not thinking in objects: OOP requires a mental shift in how to approach your programming problems compared to procedural programming. It is very easy to fall back into old habits. Try to practise abstract thinking and simplify complex real-world items into classes. It can be hard to figure out what should be an object, a class or a method at times, and it will take practice to get right.

● Linking question

Is OOP necessary for all programming or just in the modelling of complex situations? (B2)

Bank accounts

To begin, study an example of how code that uses OOP may look. The following code is based on a scenario of Bank Accounts. The process of creating variables (known within OOP as “objects”), and then performing simple tasks with them, could be enacted through the following code.

Python

```
amy = BankAccount("Amy")
brian = BankAccount("Brian")
clare = BankAccount("Clare")
# Do some transactions
amy.deposit(100)
brian.deposit(200)
clare.deposit(150)
amy.withdraw(75)
brian.deposit(75)
brian.transfer(250,clare)
# Print account info
print(amy)
print(brian)
print(clare)
```

Java

```
class Main {
    public static void main(String[] args) {
        BankAccount amy = new BankAccount("Amy");
        BankAccount brian = new BankAccount("Brian");
        BankAccount clare = new BankAccount("Clare");
        // Do some transactions
        amy.deposit(100);
        brian.deposit(200);
        clare.deposit(150);
        amy.withdraw(75);
        brian.deposit(75);
        brian.transfer(250, clare);
        // Print account info
        System.out.println(amy);
        System.out.println(brian);
        System.out.println(clare);
    }
}
```

Notice that, without having to know anything about how these objects manage their internal variables, you can understand the expected behaviour and reasonably estimate the output, which may resemble the following:

```
Account "Amy" has balance $25
Account "Brian" has balance $25
Account "Clare" has balance $400
```

This is OOP at work. It allows you to create data types and write functions that are attached to, and have access to, the information stored within.

B3.1.2 Designing classes, their methods and behaviour

Before writing code, it is important to introduce the idea of the UML Class diagram. “UML” (Unified Modelling Language) is the umbrella term for a series of standardized diagrams used within Computer Science. These have been established to provide consistency in the design of computing projects. One diagram within UML, the Class diagram, is used for articulating the design of an OOP Class.

A UML Class diagram looks like a table in three rows.

- **Row 1:** The Class name is specified in the top row for identification purposes.
- **Row 2:** The variables within each instance are listed in the second row, along with their data type.
- **Row 3:** The methods associated with the Class are listed in the third row, including their parameter signatures and return data type.

Compare the following tables to clarify. The left example provides a generic outline of the Class diagram, whereas the example on the right could be an implementation of the BankAccount Class.

Classname	BankAccount
+ field: type + field: type + field: type	– name: string – balance: float
+ method(type): type + method(type): type + method(type): type	+ BankAccount(name) + deposit(float): void + withdraw(float): void + transfer(float, BankAccount): void

■ Template and example of a UML Class diagram

Within UML Class diagrams, a final important element to note is with respect to visibility. Notice that each element is preceded by a plus or minus sign. This indicates whether the element should be set to public (plus sign) or private (minus sign). This will be discussed further as part of B3.1.5 in the topic of encapsulation.

UML Class diagrams can also be used to depict relationships when classes are dependent on one another. This is discussed further in B3.2.4.

You are expected to be able to create UML Class diagrams for classes from code, and construct code for a Class from its UML. There are practice exercises for you to complete at the end of this chapter.

Top tip!

Use your UML diagrams. UML diagrams are your blueprints. Sketch out your Classes and their relationships before writing code. This may seem unnecessary at the start, but it is a useful habit to form early that will pay dividends when you are devising increasingly large and complex projects. As you do, start with the basic classes that form the core of your project and then expand as needed. Don't try to add everything at once as you will overcomplicate it.

B3.1.4 Coding classes and instantiating objects

■ Creating the class

◆ **Instantiation:**

the line of code that declares a new object variable based on the template code provided by a class, which then executes the constructor to initialize the object.

◆ **Constructor:**

a special method within a class that is automatically executed during instantiation; its main task is to initialize any instance variables required before an instance of the object can be used by other code.

Instantiation is the process of creating a specific instance of a class, which is called an “object”. There are two key tasks performed as part of the instantiation process: the allocation of memory for the new object, and the execution of the constructor method.

The **constructor** is a special method whose role is to initialize a new object, with a specific focus on any instance variables within the object. In Java, the constructor is identified by a method having the same name as the class. In Python, the constructor is identified by the name `__init__()`. The constructor doesn't have a return type (technically, it is returning the initialized object).

In Java, the `this` keyword is used to refer to the current object instance of the class. It can be used to prefix any instance variable or method. It is particularly useful as a way of differentiating between instance variables and local variables that may have the same name. The `this` keyword is optional unless ambiguity exists, such as needing to differentiate between an instance variable and a parameter variable.

In Python, the `self` keyword performs the task of referring to the current object instance of the class. The `self` keyword is mandatory to prefix an instance variable or method. Additionally, `self` must be listed as the first parameter for all methods belonging to the class. When calling these methods in your code, you do not need to pass anything for that value; Python will do it for you.

To illustrate the syntax of constructing classes in Java and Python, the following will serve initially to construct the `BankAccount` example.

● **Top tip!**

When naming your classes, it is standard practice to use a singular noun that represents the entity the template represents. The methods (or functions) within that class should then be named as verbs, indicating the action they will perform on the objects of the class.

In addition, the naming convention dictates the following for upper- and lower-casing of names:

- **Class names:** Capitalize all words (e.g. `MyClass`, `BankAccount`)
- **Object instances:** Camel case in Java (e.g. `myObject`) and snake case in Python (e.g. `my_object`)
- **Method names:** Camel case in Java (e.g. `calculateTotal`) and snake case in Python (e.g. `calculate_total`)
- **Instance variables:** Camel case in Java and snake case in Python.

Python

```
class BankAccount:
    def __init__(self, name):
        self.name = name # Create an instance variable "name"
        # and set it to value parameter variable "name".
        self.balance = 0.0 # Set instance variable "balance" to 0.0.
    def deposit(self, amount):
        self.balance = self.balance + amount
    def withdraw(self, amount):
        self.balance = self.balance - amount
```

```
def transfer(self, amount, recipient):
    self.withdraw(amount)
    recipient.deposit(amount)
def __str__(self):
    return f"Account {self.name} has balance ${self.balance}"
```

Java

```
public class BankAccount {
    private String name; // Create an instance variable
    private double balance; // Create an instance variable
    public BankAccount(String name) {
        this.name = name // Set instance 'name' to value of parameter 'name'
        this.balance = 0.0;
    }
    public void deposit(double amount) {
        balance = balance + amount;
    }
    public void withdraw(double amount) {
        balance = balance - amount;
    }
    public void transfer(double amount, BankAccount recipient) {
        this.withdraw(amount);
        recipient.deposit(amount);
    }
    public String toString() {
        return "Account " + this.name + " has balance $" + this.balance;
    }
}
```

■ Creating an object

Creating an array is the process of declaring and instantiating an object variable based on the class type. It is at this point that you pass any parameters required by the constructor.

The following example creates an instance of an object using the identifier name `acc`, of the type `BankAccount`. You can treat `acc` just like any other variable from that point onward.

Python

```
# Python
acc = BankAccount("Neo")
print(acc)
```

Java

```
public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("Neo");
        System.out.println(acc);
    }
}
```

■ Creating an array of objects

When initializing an array of objects, it is important to note that each individual object still requires its constructor function to be executed. This may require iterating over the entire array to explicitly execute the constructor on each element. Some examples based on the BankAccount scenario follow.

● Top tip!

Because arrays contain multiple objects, their names should always be a plural of the object contained within. In this case, it could be `bankAccounts` (Java's camelCase) or `bank_accounts` (Python's snake_case).

Python

```
# Python - example 1
accounts = [
    BankAccount("Amy"),
    BankAccount("Brian"),
    BankAccount("Clare")
]

# Python - example 2
accounts = []
for i in range(0, 3):
    name = input("Name for new bank account:")
    accounts.append( BankAccount( name ) )
```

Java

```
// Java - example using static array
BankAccount[] accounts = new BankAccount[3];
accounts[0] = new BankAccount("Amy");
accounts[1] = new BankAccount("Brian");
accounts[2] = new BankAccount("Clare");

// Java - example iterating over static array
String[] names = {"Amy", "Brian", "Clare"};
BankAccount[] accounts = new BankAccounts[3];
for (int i=0; i<accounts.length; i++) {
    accounts[i] = new BankAccount( names[i] );
}

// Java - example using ArrayList
// Remember to import java.util.ArrayList
ArrayList<BankAccount> accounts = new
ArrayList<BankAccount>();
accounts.add(new BankAccount("Amy"));
accounts.add(new BankAccount("Brian"));
accounts.add(new BankAccount("Clare"));
```

B3.1.5 Encapsulation and information hiding in OOP

◆ **Encapsulation:** bundles data and the methods that manipulate that data together into a single object. It serves to hide the implementation details of the object from outside code.

◆ **Access modifiers:** the mechanisms provided by the programming language to control visibility of methods and variables within an object.

Encapsulation is the idea that programming code outside of the class should not have direct access to the data within the class. You can visualize the data as being protected by a capsule wrapped around it! The only way to permeate the capsule is through the methods the class allows.

By controlling access, encapsulation ensures that variables are only used in the manner in which they were intended, helping protect your code from invalid, error-inducing values. It also means that you can write code that uses encapsulated classes without needing to know or care how the class manages its internal state.

Successful encapsulation of your classes is fundamental to facilitating modularity in object-oriented programming.

● Linking question

How can the principles of encapsulation and information hiding (B2.5) be applied to secure network communication? (A3)

■ Access modifiers

Encapsulation is achieved through using **access modifiers**. The structure of access modifiers is different between Java and Python.

In Java, the access modifier is specified by supplying the relevant keyword in front of the variable or method definition. There are four access modifiers to know:

- **Private:** Indicates it should only be accessible by the current object.
- **Protected:** Indicates that it may be accessed by the current object, and any objects that inherit it (more on that in B3.2.1).
- **Public:** Indicates it may be accessible to any code within your program.
- **Default:** When no modifier is provided, it allows access for any object within the same Java package (as denoted by the package statement at the top of your Java file); Python does not have an equivalent of this.

In Python, access modifiers work slightly differently. They are specified through a naming convention for your variables and methods. They can be bypassed. In this sense, they are more guidelines than rules. That said, for the purposes of your course, you should avoid bypassing them.

- **Public:** The default behaviour is that variables and methods are treated as public.
- **Internal use:** The single underscore prefix (e.g. `_variable`) is used to indicate it is only for internal use within the class. While this is purely a convention, and the Python interpreter does not enforce it, most Python editors strongly hint at not using these variables by hiding them or making them less visible.
- **Name mangled:** The double underscore prefix (e.g. `__variable`) is used to instruct the Python interpreter to rename the variable or method at runtime. The name is mangled to include the class name, which makes it harder (but not impossible) to access from outside the class. For example, `__variable` in a class named `MyClass` would be mangled to `_MyClass__variable`.

This might seem like it makes a variable private, but Python doesn't have truly private variables. Name mangling was created to avoid naming conflicts, rather than to enforce strict access control. The mangled name can still be accessed from outside the class; it just requires knowledge of the name-mangling pattern Python uses.

For the purposes of your course, you can treat either the single or double underscore prefix as denoting private, but ensure you are aware of the technical subtleties involved.

■ Accessors and mutators

By setting your variables to private, the inevitable consequence is you will need to create a number of public methods through which code outside your class can interact with those variables, query their value or request they be updated.

Methods that perform these tasks are formally known as **accessors** and **mutators**. They are also often referred to as getters and setters.

For instance, if a `Person` class has variables for name and age, then there may be accessor methods `getName()` and `getAge()`, as well as mutator methods `setName()` and `setAge()`.

Crucially, the example above serves to reinforce the important role that encapsulation can play in protecting your data. You may not want it to be so easy to update a person's name in your application, and the person's age may be calculated behind the scenes using the current date and a stored date of birth, rather than just being stored as an integer that will soon drift to being out of date.

Requiring accessor and mutator methods, rather than giving external code free access to your variables, allows separation of responsibilities and keeps the class in control of what happens to its data.

◆ **Accessor:** a public method that allows external code to "access" the value of a private instance variable within an object; also known as "getter method" as it "gets" the value.

◆ **Mutator:** a public method that allows external code to update or mutate the value of a private instance variable within an object; also known as "setter method" as it "sets" the value.

● Top tip!

For beginner programmers, the following is recommended good practice:

- Set variables to **private** where possible. Create accessor and mutator methods for any attribute you want external code to have access to.
- Methods (or functions) should be **public** if you want external code to have access to them. If the method is an internal helper function, set it to **private**.

Study the updated code for the Bank Account scenario in Section B3.1 to see inclusion of access modifiers.

ACTIVITY

Research skills and thinking skills: Code analysis

Operation sabotage! Take a piece of OOP programming code and insert some deliberate errors into it that break the principle of encapsulation. Swap with a classmate, and refactor the code you are given to ensure proper use of encapsulation again. Discuss and compare changes with your classmate afterwards. Did you both spot all the errors and apply appropriate corrections?

◆ **Static:** methods and variables that belong to the class, not the individual objects. Only one copy is created that is shared with all instances in common.

B3.1.3 Static and non-static variables and methods

The term **static** is used to represent variables and methods that are associated with the class itself rather than any individual object instances.

Static variables and methods don't require an instance to be created for them to exist. They are created by the interpreter / compiler at runtime when the class is defined. This is why the main function in Java is defined as static, as it needs to exist and be executed before any objects have been created by the code itself.

Because statics do not depend on an object having been instantiated, it does mean that no static method may access instance variables within an object. It also means that any time an instantiated object makes use of a static variable or method, the objects are all accessing the same shared variable or method instead of their own instance of it. Consequently, should an object change the value of a static variable, that change will be visible to all other objects. You can think of a static variable as a global variable that is shared in common with all the objects of that class. Static variables come in useful when something needs to be shared across all objects within a class, or when the particular variable does not depend on an instance existing.

The term “non-static”, therefore, refers to the normal instance variables you have been defining within objects up until this point. Each object creates its own unique instance of these variables so, when used by one object, it does not affect the value stored in the matching variable of another object.

When referring to static variables and methods in code, it is best practice to prefix it with the class name, for example `ClassName.variableName` (see the examples in `BankAccount` below).

Using a static variable to track ID values

One common scenario for using a static variable is to ensure each object of a class has a unique ID number. An example might be product numbers for a supermarket application.

Python

```
class Product:
    next_product_id = 0 # define a static variable
    def __init__(self, name):
        self.name = name
        self.product_id = Product.next_product_id
        Product.next_product_id += 1
    def __str__(self):
        return "Product "+str(self.product_id)+": "+self.name
if __name__ == "__main__":
    products = [
        Product("Bread"),
        Product("Milk"),
        Product("Apples"),
        Product("Icecream")
    ]
    for i in range(0, len(products)):
        print(products[i])
```

Java

```
class Product {
    private static int nextProductID = 0; // define a static
    // variable
    private String name;
    private int productid;
    Product(String name) {
        this.name = name;
        this.productID = Product.nextProductID;
        Product.nextProductID++;
    }
    public String toString() {
        return "Product "+Integer.toString(this.productID)+" :
        "+this.name;
    }
}

class Main {
    public static void main(String[] args) {
        Product[] products = new Product[4];
        products[0] = new Product("Bread");
        products[1] = new Product("Milk");
        products[2] = new Product("Apples");
        products[3] = new Product("Icecream");
        for (int i=0; i<products.length; i++) {
            System.out.println( products[i] );
        }
    }
}
```

Using a static variable to track number of items in array (Java only)

In Java, another common usage of static variables is to keep track of the number of items populated within an array. Given arrays are fixed in size, the static variable can be used to ensure you place the next item at the next empty location. This is less of an issue in Python since static arrays are not really a thing, and the list construct is dynamic in size.

Alternative approaches to this would be:

- having an if statement testing for null inside a loop
- using an ArrayList, since it is dynamically resizable.

Here is an example:

Java

```
class Thing {
    private String name;
    Thing(String name) {
        this.name = name;
    }
    public String toString() {
        return this.name;
    }
}

class CollectionOfThings {
    private static int nextThing = 0; // static variable
    private Thing[] things;
    CollectionOfThings() {
        things = new Thing[100]; // Create an array of 100
        // null Things
        CollectionOfThings.nextThing = 0;
    }
    public void add(Thing t) {
        things[Collection.nextThing] = t;
        CollectionOfThings.nextThing++;
    }
    public void remove(Thing t) {
        boolean found = false;
        for (int i=0; i<CollectionOfThings.nextThing-1; i++) {
            if (things[i].equals(t)) {
                found = true;
            }
            if (found) {
                things[i] = things[i+1];
            }
        }
        if (found) {
            CollectionOfThings.nextThing--;
        }
    }
    public void printAll() {
        for (int i=0; i<CollectionOfThings.nextThing; i++) {
            System.out.println(things[i]);
        }
    }
}
```

Top tip!

Remember that static means one shared item for the class, not individual instances. Only use static for things that are common to all objects (like a company name), and use normal, non-static for properties specific to an instance (like an employee number). Experiment with writing code snippets to see the effect of changing members from static to non-static, and vice versa.

Linking question

In what ways can OOP (B1) be applied to database (A3) development?

B3.1 End-of-section examples

ACTIVITY

Social skills: Paired programming

Paired programming is a research-based approach to help develop confidence in beginner programmers. Work in pairs to design and implement your programming projects, with one student writing the code while the other reviews it and suggests improvements, switching roles periodically. Consider the analogy of having both a driver and a navigator at the computer keyboard.

■ Bank accounts

See below for the full code for the Bank Accounts scenario, updated to illustrate initialization of an array, use of access modifiers and static with non-static variables and methods.

Notice two static variables and one static method have been implemented:

- `interest_rate`: As all bank accounts will have the same interest rate, it makes sense for this to be static. That way, it only needs updating once, and all bank accounts will automatically apply the change.
- `next_account_number`: By checking and incrementing this value every time an object is created, it can ensure that no two bank accounts have the same account number.
- `find()`: Provides functionality to search an array of `BankAccounts` to find one of the requested name (supplied through the parameters).

Python

```
class BankAccount:
    interest_rate = 5.00 # Static variable
    next_account_number = 1001 # Static variable
    def __init__(self, name):
        self._name = name
        self._balance = 0
        # Use the class name as a prefix to access the static variables
        self._account_number = BankAccount.next_account_number
        BankAccount.next_account_number += 1
    def deposit(self, amount):
        self._balance += amount
    def withdraw(self, amount):
        if self._balance < amount:
            return False
        self._balance -= amount
        return True
    def transfer(self, amount, recipient):
        if self.withdraw(amount):
            recipient.deposit(amount)
            return True
        else:
            return False
```

```

def apply_interest(self):
    self._balance += self._balance * (BankAccount.interest_rate / 100)
def get_balance(self):
    return self._balance
def get_name(self):
    return self._name
def __str__(self):
    return f"Account {self._account_number}: {self._name} has balance
${self._balance}"
    @staticmethod
    def find(accounts, name):
        for acc in accounts:
            if acc.get_name() == name:
                return acc
        return None # No matching account found
# Example usage:
if __name__ == "__main__":
    # Initialize a list with 3 accounts
    accounts = [BankAccount("Amy"), BankAccount("Brian"), BankAccount("Clare")]
    # Perform some transactions
    accounts[0].deposit(500)
    accounts[1].deposit(500)
    accounts[2].deposit(500)
    accounts[0].transfer(150, accounts[2])
    # Apply interest to all account balances
    for acc in accounts:
        acc.apply_interest()
    # Perform transaction using a static method to search the list
    amy = BankAccount.find(accounts, "Amy")
    if amy:
        amy.deposit(42)
    # Print balances using the __str__ method
    for acc in accounts:
        print(acc)

```

Java

```

public class BankAccount {
    private static double interestRate = 5.00; // static variable
    private static int nextAccountNumber = 1001; // static variable
    private String name;
    private double balance;
    private int accountNumber;
}

```

```

public BankAccount(String name) {
    this.name = name;
    this.balance = 0;
    // Use the class name as a prefix to access the static variables
    this.accountNumber = BankAccount.nextAccountNumber;
    BankAccount.nextAccountNumber++;
}
public void deposit(double amount) {
    balance += amount;
}
public boolean withdraw(double amount) {
    if (balance < amount) {
        return false;
    }
    balance -= amount;
    return true;
}
public boolean transfer(double amount, BankAccount recipient) {
    if (withdraw(amount)) {
        recipient.deposit(amount);
        return true;
    } else {
        return false;
    }
}
public void applyInterest() {
    // balance is instance variable, interestRate is static variable
    balance += balance * (BankAccount.interestRate/100);
}
public double getBalance() {
    // Since balance is private, this public function will allow external
    // code to check the balance
    return balance;
}
public String getName() {
    // Since name is private, this public function will allow external code
    // to get the name
    return name;
}
public String toString() {
    return "Account "+accountNumber+": "+name+" has balance $" +balance;
}
public static BankAccount find(BankAccount[] accounts, String name) {
    for (BankAccount acc : accounts) {
        if (acc.getName().equals(name)) {
            return acc;
        }
    }
}

```

```

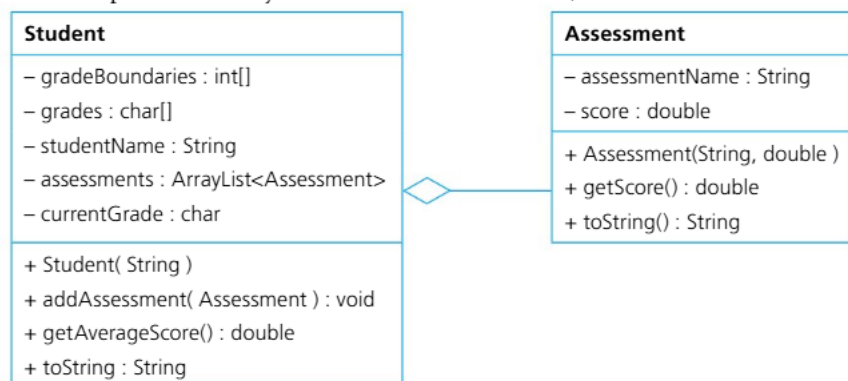
    }
    return null; // No matching account found;
}

public static void main(String[] args) {
    // Initialize an array with 3 accounts
    BankAccount[] accounts = new BankAccount[10];
    accounts[0] = new BankAccount("Amy");
    accounts[1] = new BankAccount("Brian");
    accounts[2] = new BankAccount("Clare");
    // Perform some transactions using array index
    accounts[0].deposit(500);
    accounts[1].deposit(500);
    accounts[2].deposit(500);
    accounts[0].transfer(150, accounts[2]);
    // Apply interest to all account balances
    for (int i=0; i<3; i++) {
        accounts[i].applyInterest();
    }
    // Perform transaction using a static function to search the array
    BankAccount amy = BankAccount.find(accounts, "Amy");
    if (amy != null) {
        amy.deposit(42);
    }
    // Print balances using the toString() function
    for (int i=0; i<3; i++) {
        System.out.println(accounts[i]);
    }
}
}

```

■ Students and grades

Here is another complete example, this time also showing the use of objects as instance variables within other objects. In this case, each Student object contains an array of Grade objects. Study the example and identify the use of access modifiers, static and non-static variables.



■ UML diagram of Student and Assessment Classes

Python

```
class Assessment:
    def __init__(self, assessment_name, score):
        self._assessment_name = assessment_name
        self._score = score
    def get_score(self):
        return self._score
    def __str__(self):
        return f"{self._assessment_name}: {self._score}%"

class Student:
    _grade_boundaries = [80, 65, 50, 35, 20]
    _grades = ["A", "B", "C", "D", "E"]
    def __init__(self, student_name):
        self.student_name = student_name
        self._assessments = []
        self.current_grade = "N" # Default to N before any grades are added
    def add_assessment(self, assessment):
        self._assessments.append(assessment)
        # Calculate new average
        average = self._get_average_score()
        # Update current grade
        self.current_grade = "N" # Start with the default
        for i in range(0, len(Student._grade_boundaries)):
            if average >= Student._grade_boundaries[i]:
                self.current_grade = Student._grades[i]
                break
    def _get_average_score(self):
        if not self._assessments:
            return 0
        total = 0
        for assessment in self._assessments:
            total += assessment.get_score()
        return total / len(self._assessments)
    def __str__(self):
        return f"Student {self.student_name} has grade {self.current_grade}"

if __name__ == "__main__":
    student = Student("Doris")
    student.add_assessment(Assessment("Test 1", 75))
    student.add_assessment(Assessment("Homework", 85))
    student.add_assessment(Assessment("Exam", 65))
    print(student)
```

Java

```
import java.util.ArrayList;
class Assessment {
    private String assessmentName;
    private double score; // Score as a percentage
    public Assessment(String assessmentName, double score) {
        this.assessmentName = assessmentName;
        this.score = score;
    }
    public double getScore() {
        return score;
    }
    public String toString() {
        return assessmentName + ": " + score;
    }
}
class Student {
    private static final int[] gradeBoundaries = { 80, 65, 50, 35, 20 };
    // Represents A, B, C, D, E
    private static final char[] grades = { 'A', 'B', 'C', 'D', 'E' };
    private String studentName;
    private ArrayList<Assessment> assessments;
    private char currentGrade;
    public Student(String studentName) {
        this.studentName = studentName;
        this.assessments = new ArrayList<>();
        this.currentGrade = 'N'; // Default to N before any grades are added
    }
    public void addAssessment(Assessment assessment) {
        assessments.add(assessment);
        // Calculate new average
        double average = getAverageScore();
        // Update current grade
        currentGrade = 'N'; // Start with the default
        for (int i = 0; i < Student.gradeBoundaries.length; i++) {
            if (average >= Student.gradeBoundaries[i]) {
                currentGrade = Student.grades[i];
                break;
            }
        }
    }
    private double getAverageScore() {
        if (assessments.isEmpty()) {
            return 0;
        }
        double sum = 0;
```

```

        for (Assessment assessment : assessments) {
            sum += assessment.getScore();
        }
        return sum / assessments.size();
    }
    @Override
    public String toString() {
        return "Student " + studentName + " has grade " + currentGrade;
    }
}
public class Main {
    public static void main(String[] args) {
        Student student = new Student("Doris");
        student.addAssessment(new Assessment("Test 1", 75));
        student.addAssessment(new Assessment("Homework", 85));
        student.addAssessment(new Assessment("Exam", 65));
        System.out.println(student);
    }
}

```

ACTIVITY

Self-management skills and thinking skills: Extend one of these sample projects

Using one of the sample projects provided as a starting point, add methods and variables to provide supplemental functionality to the project. Create a checklist of skills you want to add to the project, perhaps including the following suggestions:

- Reading and writing data to files so data is not lost between program execution.
- Adding a console user interface to provide interactivity and the capacity to use the program to perform different tasks.
- Using a combination of static arrays and dynamic lists, as well as static and non-static OOP methods and variables.

REVIEW QUESTIONS

- 1 Which of the following is NOT an advantage of using OOP?
 - a Reusability of code
 - b Easy to debug
 - c Improved performance in all scenarios
 - d Encapsulation of data
- 2 What does abstraction in OOP help with?
 - a Removing all bugs from an application
 - b Hiding complex implementations behind simpler interfaces
 - c Decreasing the use of memory
 - d Making code public to all classes

- 3 In UML Class diagrams, there is a three-row box used to represent a Class. What does the second row contain?
 - a Class name
 - b Methods
 - c Properties
 - d Statics
- 4 Which of the following is true about methods in a Class diagram?
 - a They are depicted with a minus sign (–) for private methods
 - b They cannot accept parameters
 - c They must be static
 - d They are optional
- 5 Which statement is true about static methods in Java?
 - a They can be called on instances of a Class
 - b They are called on the Class itself, not the instance
 - c They can directly access and modify instance variables
 - d They must return a value
- 6 What is a non-static variable also known as?
 - a Class variable
 - b Local variable
 - c Global variable
 - d Instance variable
- 7 What is the purpose of a constructor in a Class?
 - a To declare variables
 - b To initialize an object
 - c To clean up resources
 - d To return data
- 8 Which of the following declarations correctly creates an array of objects in Java?
 - a `ClassName[] arrayName = new ClassName[5];`
 - b `ClassName arrayName[] = new ClassName(5);`
 - c `ClassName arrayName = new ClassName[5];`
 - d `ClassName[] arrayName = new ClassName();`
- 9 Which of the following best describes encapsulation?
 - a Storing data in public fields
 - b Combining data and methods that operate on the data into a single unit or Class
 - c Dividing code into various functions
 - d Making all methods static
- 10 What are access modifiers used in OOP for?
 - a To define how variables can be modified
 - b To name methods and variables
 - c To indicate static methods only
 - d To control the visibility of Class members
- 11 Describe one disadvantage of using OOP.
- 12 Explain how encapsulation can benefit a software project.
- 13 Explain why a static method cannot access non-static fields or methods.
- 14 Explain the role of the new keyword in object creation.
- 15 Describe an example of how private access modifiers contribute to information hiding.

PROGRAMMING EXERCISES

1 Library system

Create a simple library system where each book can be either available or borrowed. Implement methods to manage the state of each book and track the total count of books in the library.

Java

```
class Book {
    private static int bookCount = 0;
    private String title;
    private boolean isBorrowed;
    public Book(String title) {
        this.title = title;
        this.isBorrowed = false;
        bookCount++;
    }
    public void borrow() {
        if (!isBorrowed) {
            isBorrowed = true;
            System.out.println(title + " has been borrowed.");
        } else {
            System.out.println(title + " is already borrowed.");
        }
    }
    public void returnBook() {
        // Implement logic to mark the book as not borrowed
    }
    public static int getBookCount() {
        return bookCount;
    }
    public String toString() {
        return title + " - " + (isBorrowed ? "Borrowed" : "Available");
    }
}

public class Library {
    public static void main(String[] args) {
        Book[] books = new Book[5];
        books[0] = new Book("Java Fundamentals");
        books[1] = new Book("The Art of Computer Programming");
        // Add more books and implement borrowing logic
    }
}
```

- a Complete the returnBook method.
- b Instantiate more books and simulate borrowing and returning books.
- c Print the status of all books and the total book count.

2 Simple inventory system

Create a basic inventory management system for a store. Each product has an ID, name, price and quantity. Implement functionality to add products, update inventory quantity and list all products.

Java

```
class Product {
    private static int nextProductId = 100;
    private int productId;
    private String name;
    private double price;
    private int quantity;
    public Product(String name, double price, int quantity) {
        this.productId = nextProductId++;
        this.name = name;
        this.price = price;
        this.quantity = quantity;
    }
    public void updateQuantity(int amount) {
        // Implement logic to update product quantity
    }
    @Override
    public String toString() {
        return "Product{" +
            "productId=" + productId +
            ", name='" + name + '\'' +
            ", price=" + price +
            ", quantity=" + quantity +
            "}";
    }
}

class Inventory {
    private Product[] products;
    private int size;
    public Inventory(int capacity) {
        products = new Product[capacity];
        size = 0;
    }
    public boolean addProduct(Product product) {
        if (size < products.length) {
            products[size] = product;
            size++;
            return true;
        }
        return false;
    }
}
```

```

    public void listProducts() {
        for (Product product : products) {
            if (product != null) {
                System.out.println(product);
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Inventory inventory = new Inventory(100);
        inventory.addProduct(new Product("Laptop", 999.99, 10));
        inventory.addProduct(new Product("Smartphone", 499.99, 20));
        // Update quantities and list inventory
    }
}

```

- a Implement the updateQuantity method in the Product class to adjust the stock of a product.
- b Test adding products, updating quantities and listing all products to ensure the inventory displays correctly.